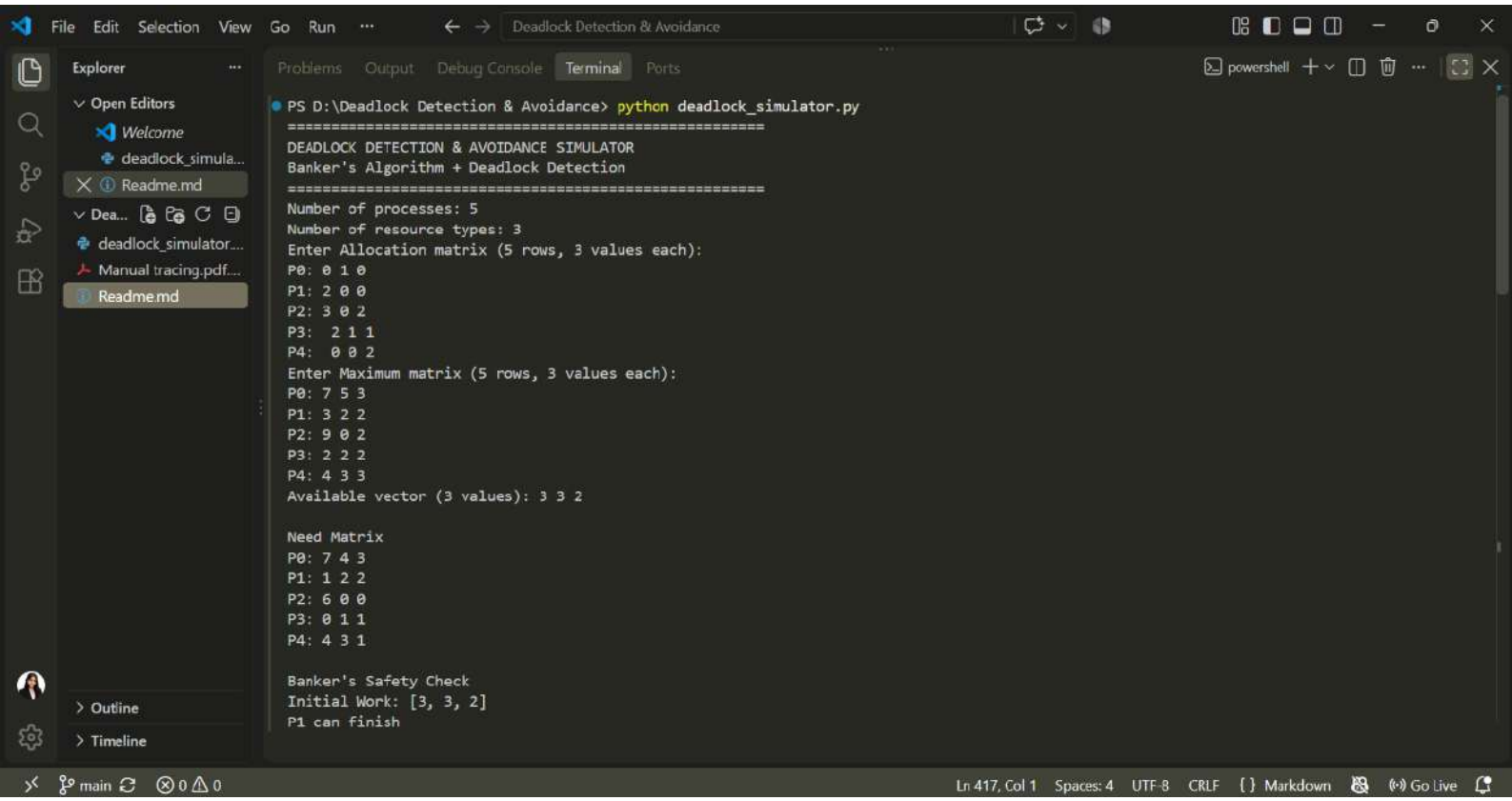
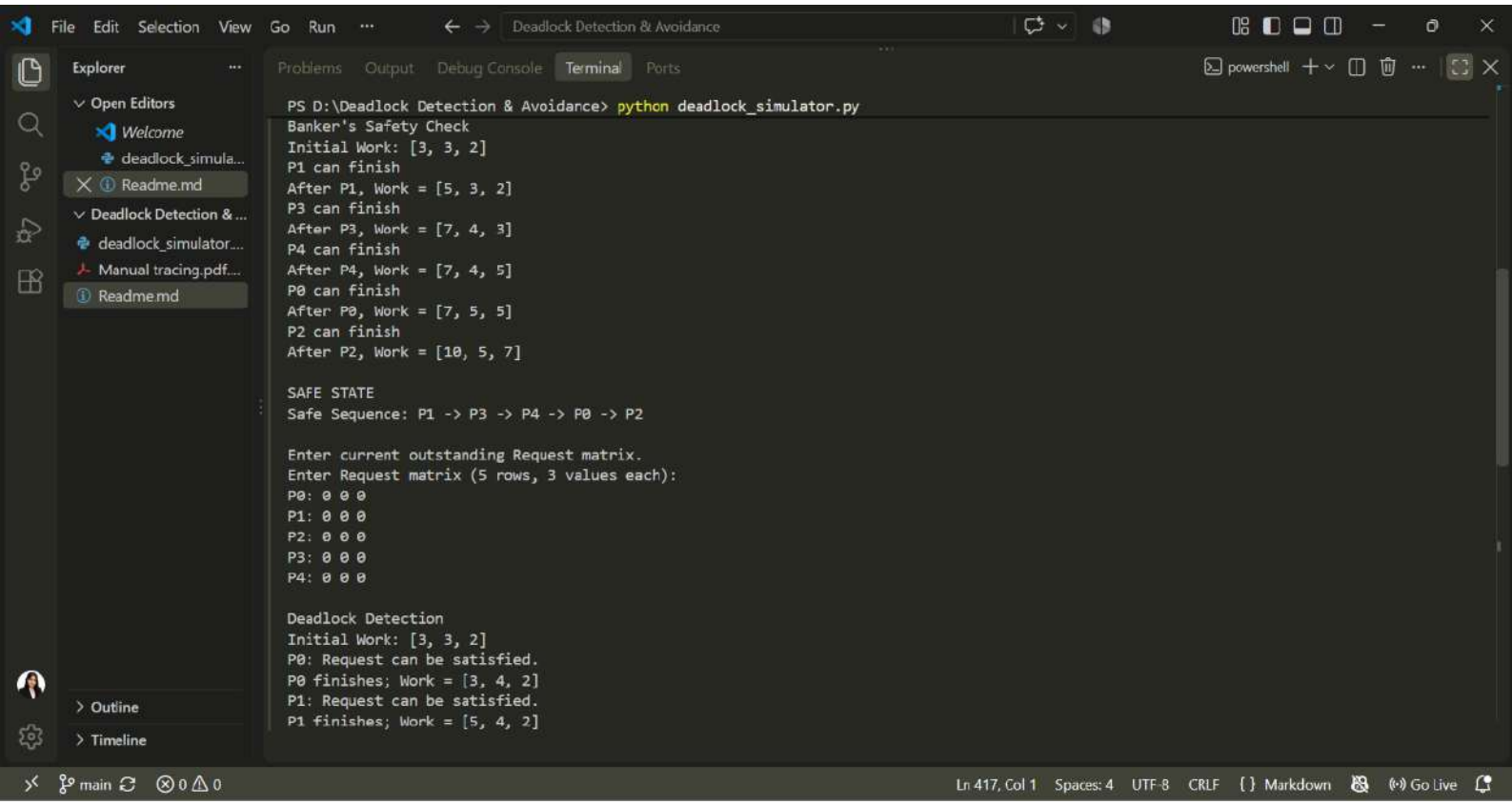
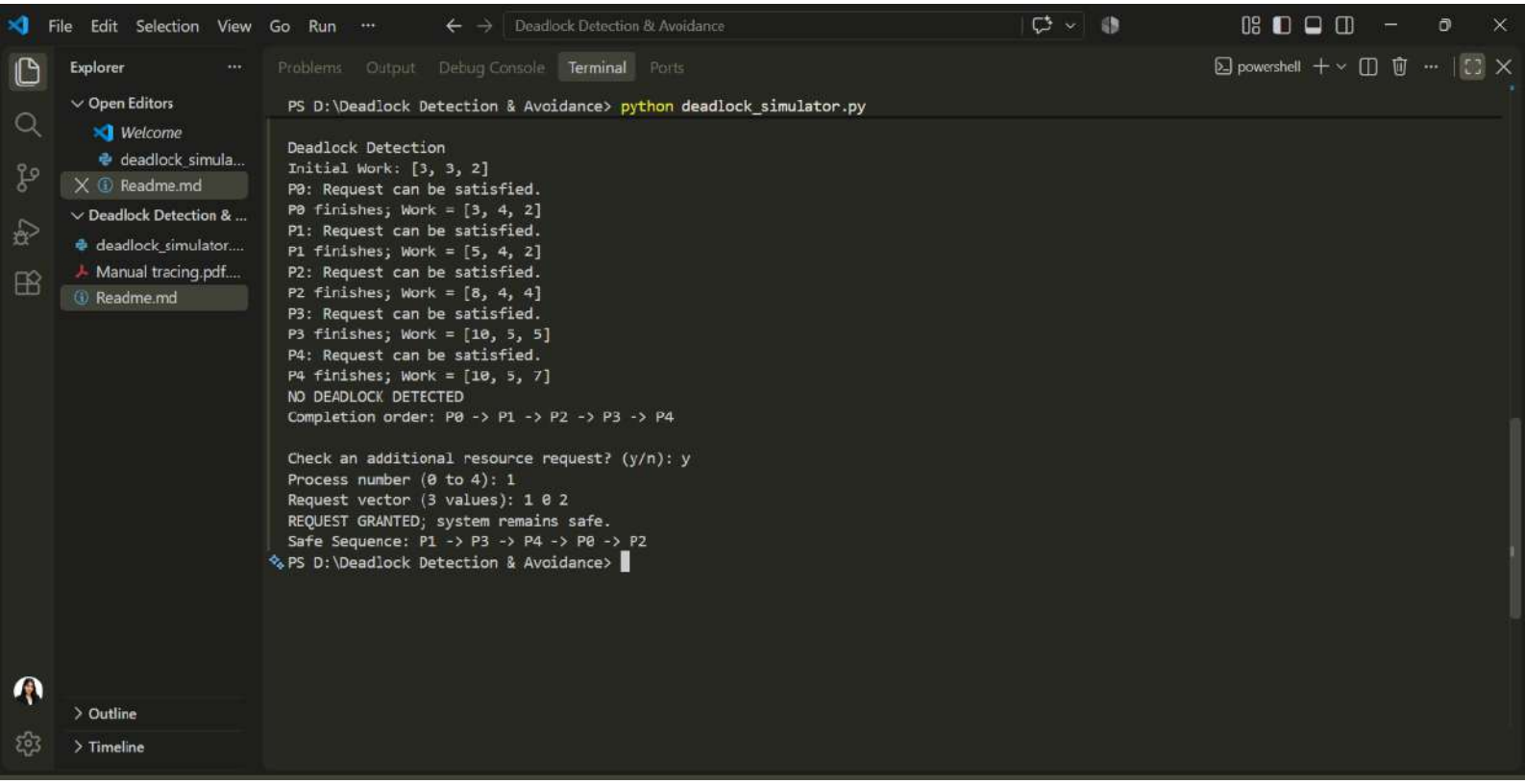

File Edit Selection View Go Run ···
Deadlock Detection & Avoidance
Explorer
Open Editors
Welcome
deadlock_simula...
Readme.md
Dea...
deadlock_simulator....
Manual tracing.pdf....
Readme.md
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
PS D:\Deadlock Detection & Avoidance> python deadlock_simulator.py
==========
DEADLOCK DETECTION & AVOIDANCE SIMULATOR
Banker's Algorithm + Deadlock Detection
==========
Number of processes: 5
Number of resource types: 3
Enter Allocation matrix (5 rows, 3 values each):
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2
Enter Maximum matrix (5 rows, 3 values each):
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3
Available vector (3 values): 3 3 2
Need Matrix
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1
Banker's Safety Check
Initial Work: [3, 3, 2]
P1 can finish
main
Ln 417, Col 1 Spaces: 4 UTF-8 CRLF Markdown Go Live


File Edit Selection View Go Run ···
Deadlock Detection & Avoidance
Explorer
Open Editors
Welcome
deadlock_simula...
Readme.md
Deadlock Detection & ...
deadlock_simulator....
Manual tracing.pdf....
Readme.md
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
PS D:\Deadlock Detection & Avoidance> python deadlock_simulator.py
Banker's Safety Check
Initial Work: [3, 3, 2]
P1 can finish
After P1, Work = [5, 3, 2]
P3 can finish
After P3, Work = [7, 4, 3]
P4 can finish
After P4, Work = [7, 4, 5]
P0 can finish
After P0, Work = [7, 5, 5]
P2 can finish
After P2, Work = [10, 5, 7]
SAFE STATE
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
Enter current outstanding Request matrix.
Enter Request matrix (5 rows, 3 values each):
P0: 0 0 0
P1: 0 0 0
P2: 0 0 0
P3: 0 0 0
P4: 0 0 0
Deadlock Detection
Initial Work: [3, 3, 2]
P0: Request can be satisfied.
P0 finishes; Work = [3, 4, 2]
P1: Request can be satisfied.
P1 finishes; Work = [5, 4, 2]
main
0 0
Ln 417, Col 1 Spaces: 4 UTF-8 CRLF { } Markdown
Go Live


File Edit Selection View Go Run ···
Deadlock Detection & Avoidance
Explorer
Open Editors
Welcome
deadlock_simula...
Readme.md
Deadlock Detection & ...
deadlock_simulator....
Manual tracing.pdf....
Readme.md
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
PS D:\Deadlock Detection & Avoidance> python deadlock_simulator.py
Deadlock Detection
Initial Work: [3, 3, 2]
P0: Request can be satisfied.
P0 finishes; Work = [3, 4, 2]
P1: Request can be satisfied.
P1 finishes; Work = [5, 4, 2]
P2: Request can be satisfied.
P2 finishes; Work = [8, 4, 4]
P3: Request can be satisfied.
P3 finishes; Work = [10, 5, 5]
P4: Request can be satisfied.
P4 finishes; Work = [10, 5, 7]
NO DEADLOCK DETECTED
Completion order: P0 -> P1 -> P2 -> P3 -> P4
Check an additional resource request? (y/n): y
Process number (0 to 4): 1
Request vector (3 values): 1 0 2
REQUEST GRANTED; system remains safe.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
PS D:\Deadlock Detection & Avoidance>